INSTITUTO FEDERAL
Goiás

Deep Learning
Lidando com (Hiper)parâmetros

Prof. Sérgio Canuto - sergio.canuto@ifg.edu.br

Tópicos:

INSTITUTO FEDERAL
Goiás

- Implementação (keras)
- Tratando redes profundas
- Problemas…
    - Potenciais soluções/parametrizações
- Interpretação e Correção de Resultados
- Depurando código com tensorboard.

In [1]:
```
from tensorflow import keras
```

Base de Dados usada Fashion MNIST. 70.000 imagens em escala de cinza, com 784 features (28x28). Keras disponibiliza diversas bases de dados para tratamento. ¶

In [2]:
```
fashion_mnist = keras.datasets.fashion_mnist
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist.load_data()
```

Diferente do Scikit-Learn, as imagens estão no formato 28x28 e não em uma dimensão. Ainda, o tipo de dado é inteiro.

In [3]:
```
print (X_train_full.shape, X_train_full.dtype)
```

```
(60000, 28, 28) uint8
```

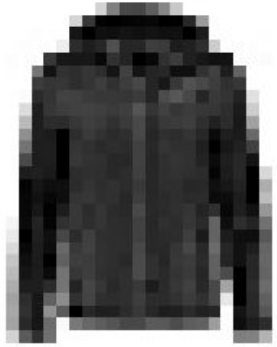
Embora o treino esteja dividido em treino e teste, vamos ter que criar o conjunto de validação.

Outra coisa importante é converter os dados para escala 0-1

In [4]:
```
X_valid, X_train = X_train_full[:5000] / 255., X_train_full[5000:] / 255.
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]
X_test = X_test / 255.
```

Vamos ver uma instância:

In [5]:
```
import matplotlib.pyplot as plt
plt.imshow(X_train[0], cmap="binary")
plt.axis('off')
plt.show()
```



In [6]:
```
y_train
```

Out[6]:
```
array([4, 0, 7, ..., 3, 0, 5], dtype=uint8)
```

Nesse caso, cada classe se refere a uma peça de roupa

In [7]:
```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

In [8]:
```
class_names[y_train[0]]
```

Out[8]:
```
'Coat'
```

Vamos ver uma amostra das instâncias, 40 delas...

In [9]:
```
n_rows = 10
n_cols = 10
plt.figure(figsize=(n_cols * 1.2, n_rows * 1.2))
for row in range(n_rows):
    for col in range(n_cols):
        index = n_cols * row + col
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(X_train[index], cmap="binary", interpolation="nearest")
        plt.axis('off')
        plt.title(class_names[y_train[index]], fontsize=12)
plt.subplots_adjust(wspace=0.2, hspace=0.5)
save_fig('fashion_mnist_plot', tight_layout=False)
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-9-15d96644bf85> in <module>
     10         plt.title(class_names[y_train[index]], fontsize=12)
     11 plt.subplots_adjust(wspace=0.2, hspace=0.5)
---> 12 save_fig('fashion_mnist_plot', tight_layout=False)
     13 plt.show()

NameError: name 'save_fig' is not defined
```



Vamos então construir uma Rede Neural

```
model = keras.models.Sequential()
model.add(keras.layers.Flatten(input_shape=[28, 28]))
model.add(keras.layers.Dense(300, activation="relu"))
model.add(keras.layers.Dense(100, activation="relu"))
model.add(keras.layers.Dense(10, activation="softmax"))
```

A primeira camada transforma a entrada em um vetor de uma dimensão. Essa primeira camada não tem pesos, somente as entradas

As demais camadas são densas pois atribuem os pesos para camada anterior e os neurônios da camada. Vejam o parâmetro para função de ativação, ReLU.

A camada densa tem sua matriz de pesos e o vetor de bias. Vejam que uma camada densa calcula para cada neurônio a função h(X)=FunçaoAtivaçao(XW+b)

A última camada possuem 10 neurônios pois estamos tratando de 10 classes exclusivas.

A função summary descreve a rede criada.

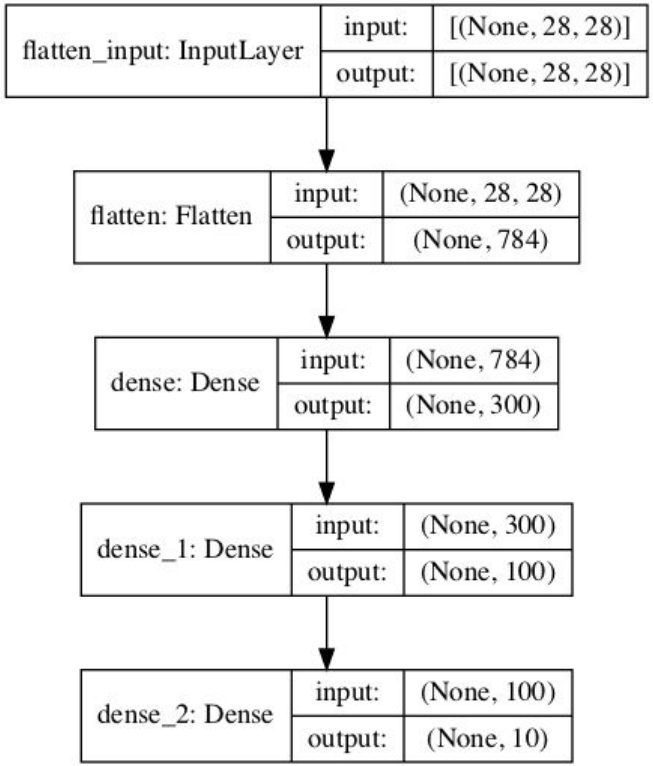
```
]: model.summary()
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 300)               235500
_________________________________________________________________
dense_1 (Dense)              (None, 100)               30100
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1010
=================================================================
Total params: 266,610
Trainable params: 266,610
Non-trainable params: 0
_________________________________________________________________
```

Note que 23550 parâmetros se refere aos pesos e bias que serão aprendidos, 784x300 no caso da camda "Dense".

Uma representação gráfica dessa rede pode ser obtida:

In [22]:
```
keras.utils.plot_model(model, "my_fashion_mnist_model.png", show_shapes=True)
```



Os valores dos pesos e bias em cada camada podem ser acessados;

In [23]:
```
model.layers
```

Out[23]:
```
[<tensorflow.python.keras.layers.core.Flatten at 0x1685be25d08>,
 <tensorflow.python.keras.layers.core.Dense at 0x1685be25c08>,
 <tensorflow.python.keras.layers.core.Dense at 0x1685be54d88>,
 <tensorflow.python.keras.layers.core.Dense at 0x1685c077bc8>]
```

In [24]:
```
hidden1 = model.layers[1]
hidden1.name
```

Out[24]:
```
'dense'
```

In [25]:
```
weights, biases = hidden1.get_weights()
```

In [26]:
```
print ("Olá, eu sou o conhecimento:", weights)
```

```
Olá, eu sou o conhecimento: [[-0.01982554 -0.05972073 -0.07404873 ... -0.070458   -0.06915936
   0.06313017]
 [-0.01207474 -0.02712235 -0.06105417 ...  0.02781773 -0.04483516
   0.0124708 ]
 [ 0.05548631  0.03520936 -0.03441961 ...  0.03328292  0.02233611
   0.06562936]
 ...
 [-0.04838773  0.06220061  0.03437502 ...  0.02119237  0.05427265
   0.04171459]
 [-0.03504748 -0.01263107 -0.02968967 ...  0.02076242 -0.07128673
   0.02752175]
 [ 0.00569931  0.0725559   0.07225588 ... -0.06055948  0.02769873
   0.01649418]]
```

In [31]:
```
biases
```

Out[31]:
```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
       
```

Compilando...

In [28]:
```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd",
              metrics=["accuracy"])
```

Rodando...

In [*]:
```
history = model.fit(X_train, y_train, epochs=30,
                    validation_data=(X_valid, y_valid))
```

```
Epoch 1/30
1719/1719 [==============================] - 6s 4ms/step - loss: 0.7131 - accuracy: 0.7634 - val_loss: 0.5110 - val_accuracy:
0.8228
Epoch 2/30
1719/1719 [==============================] - 7s 4ms/step - loss: 0.4885 - accuracy: 0.8284 - val_loss: 0.4431 - val_accuracy:
0.8478
Epoch 3/30
1719/1719 [==============================] - 6s 4ms/step - loss: 0.4432 - accuracy: 0.8453 - val_loss: 0.4522 - val_accuracy:
0.8382
Epoch 4/30
1719/1719 [==============================] - 7s 4ms/step - loss: 0.4173 - accuracy: 0.8531 - val_loss: 0.3873 - val_accuracy:
0.8650
Epoch 5/30
```

```
Epoch 25/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2480 - accuracy: 0.9108 - val_loss: 0.3021 - val_accuracy:
0.8930
Epoch 26/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2441 - accuracy: 0.9122 - val_loss: 0.3201 - val_accuracy:
0.8836
Epoch 27/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2403 - accuracy: 0.9134 - val_loss: 0.2899 - val_accuracy:
0.8966
Epoch 28/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2354 - accuracy: 0.9150 - val_loss: 0.3014 - val_accuracy:
0.8926
Epoch 29/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2318 - accuracy: 0.9158 - val_loss: 0.3116 - val_accuracy:
0.8920
Epoch 30/30
1719/1719 [==============================] - 6s 3ms/step - loss: 0.2279 - accuracy: 0.9182 - val_loss: 0.3153 - val_accuracy:
0.8846
```

Veja que accuracy e val_accuracy te ajuda a verificar o overfitting.

Em casos em que a Base de Dados está muito desbalanceada, é útil a utilização do class_weith

Em caso de instâncias que tenham sido rotuladas por pessoas, e merecem mais peso, é possível usar sample_weight.

O método fit retorna um objeto HISTORY com parâmetros de treinament (history.params), epchs executados (history.epoch), e loss e métrica avaliada. Vejam como podemos observar a execução da rede.

In [37]:
```
history.history.keys()
```

Out[37]:
```
dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

O método fit retorna um objeto HISTORY com parâmetros de treinament (history.params), epchs executados (history.epoch), e loss e métrica avaliada. Vejam como podemos observar a execução da rede.

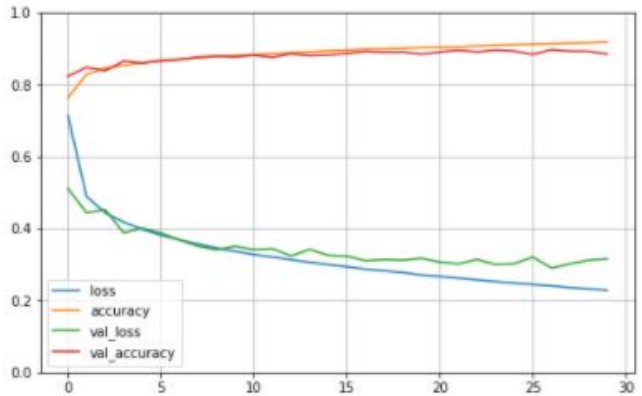
In [30]:
```
history.history.keys()
```

Out[30]:
```
dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

In [31]:
```
import pandas as pd
pd.DataFrame(history.history).plot(figsize=(8, 5))
plt.grid(True)
plt.gca().set_ylim(0, 1)
save_fig("keras_learning_curves_plot")
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-31-d3ce0e3ea8e7> in <module>
      3 plt.grid(True)
      4 plt.gca().set_ylim(0, 1)
----> 5 save_fig("keras_learning_curves_plot")
      6 plt.show()

NameError: name 'save_fig' is not defined
```



Após avaliar seu método com a validação, é possível avaliar no teste.

In [32]:
```
model.evaluate(X_test, y_test)
```

```
313/313 [==============================] - 1s 3ms/step - loss: 0.3556 - accuracy: 0.8707
```

Out[32]:
```
[0.3556255102157593, 0.8707000017166138]
```

Fazendo predição de 3 imagens

In [33]:
```
X_new = X_test[:3]
y_proba = model.predict(X_new)
y_proba.round(2)
```

Out[33]:
```
array([[0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.01, 0.  , 0.99],
       [0.  , 0.  , 1.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ],
       [0.  , 1.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ]],
      dtype=float32)
```

In [34]:
```
y_pred = model.predict_classes(X_new)
y_pred
```

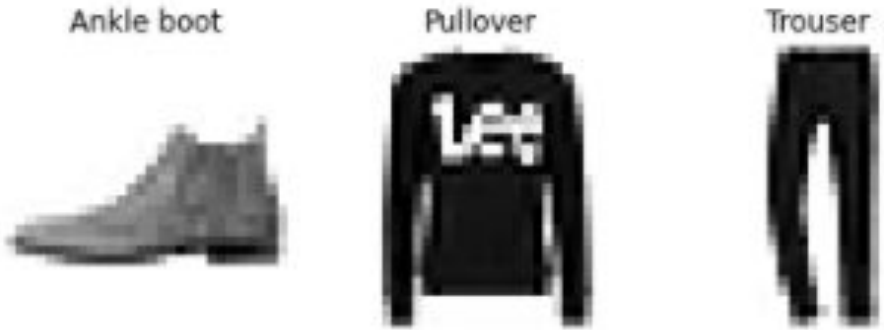
```
WARNING:tensorflow:From <ipython-input-34-81ace37e545f>:1: Sequential.predict_classes (from tensorflow.python.keras.engine.sequential) is deprecated and will be removed after 2021-01-01.
Instructions for updating:
Please use instead:* `np.argmax(model.predict(x), axis=-1)`,   if your model does multi-class classification   (e.g. if it uses a `softmax` last-layer activation).* `(model.predict(x) > 0.5).astype("int32")`,   if your model does binary classification   (e.g. if it uses a `sigmoid` last-layer activation).
```

Out[34]:
```
array([9, 2, 1], dtype=int64)
```

In [35]:
```
import numpy as np
np.array(class_names)[y_pred]
```

Out[35]:
```
array(['Ankle boot', 'Pullover', 'Trouser'], dtype='<U11')
```

In [37]:
```
y_new = y_test[:3]
plt.figure(figsize=(7.2, 2.4))
for index, image in enumerate(X_new):
    plt.subplot(1, 3, index + 1)
    plt.imshow(image, cmap="binary", interpolation="nearest")
    plt.axis('off')
    plt.title(class_names[y_test[index]], fontsize=12)
plt.subplots_adjust(wspace=0.2, hspace=0.5)
plt.show()
```



Treinando Redes Profundas… Pré-processando

- Formas mais usadas:
  - re-escalar entre 0 e 1
  - re-escalar entre -1 e 1
  - Intuitivamente…
    - Se a sua função de ativação da camada de entrada tem uma saída entre 0-1, melhor re-escalar entre 0-1
    - Um range muito largo de valores (fora de 0-1) implica em maior esforço para ajustar os pesos da primeira camada
    - Espalhamento muito fragmentado com features em diferentes ranges implica em maior esforço para ajustar os pesos da primeira camada

Pré-processamento - Transformação dos Dados

- Formas mais comuns de normalização:
  - escalonamento decimal
  - max-min
  - z-score

Pré-processamento - Transformação dos Dados

- Transformação dos dados: Escalonamento decimal

Normalização por escalonamento decimal

Transforma os valores para decimais, da seguinte forma:

$$v' = \frac{v}{10^j} \tag{7}$$

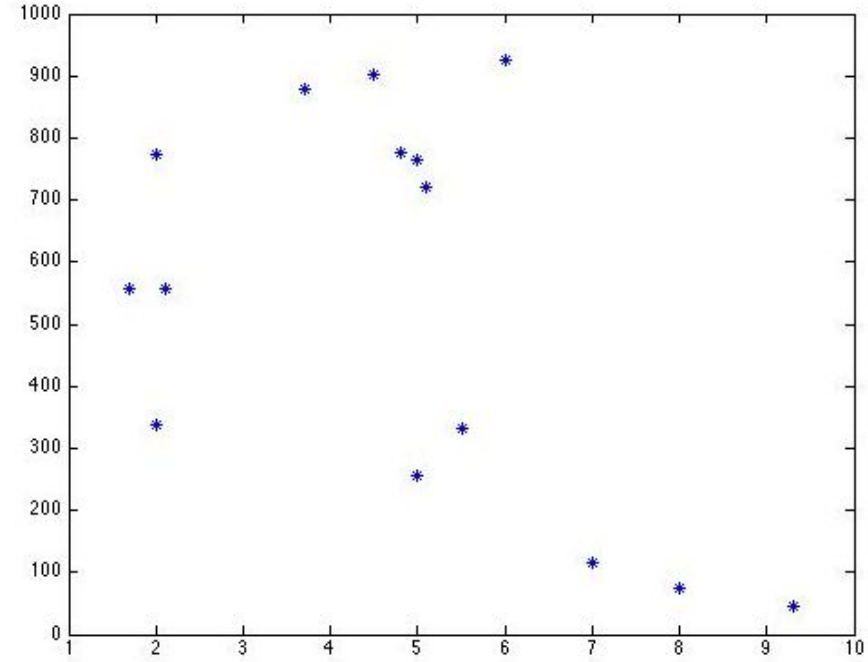
onde j é igual a 1 se o maior valor absoluto no conjunto de valores do atributo A é < 10, é igual a 2 se o maior valor absoluto no conjunto de valores do atributo A é ≥ 10 e < 100, e assim por diante.

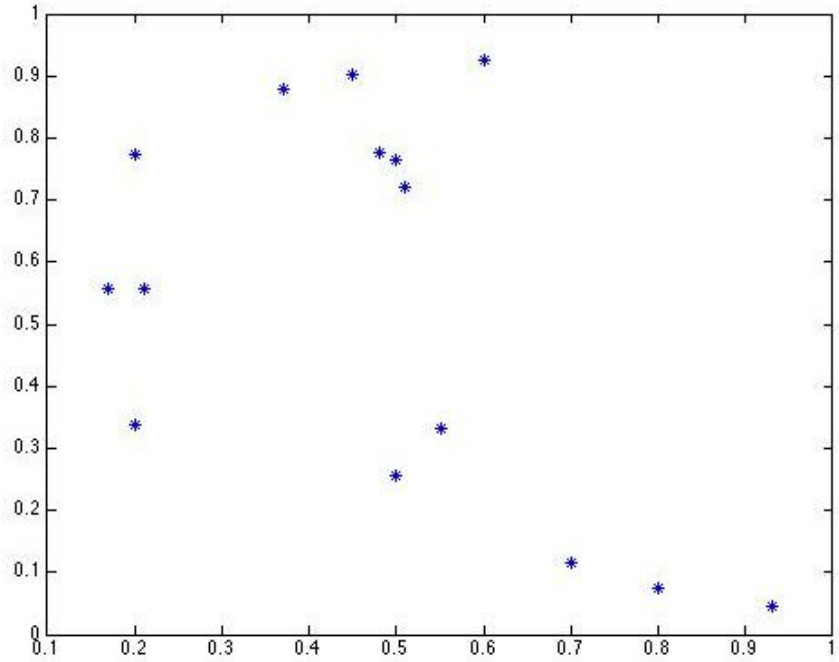
Essa normalização pode alterar os relacionamentos originais entre os exemplares.

O valor j precisam ser armazenado para que possa ser usado na normalização de novos exemplares.

Pré-processamento - Transformação dos Dados

- Transformação dos dados: Escalonamento decimal





Pré-processamento - Transformação dos Dados

- Transformação dos dados: Normalização Max-min
  - Mantém os valores dentro do intervalo [novo_min, novo_max]

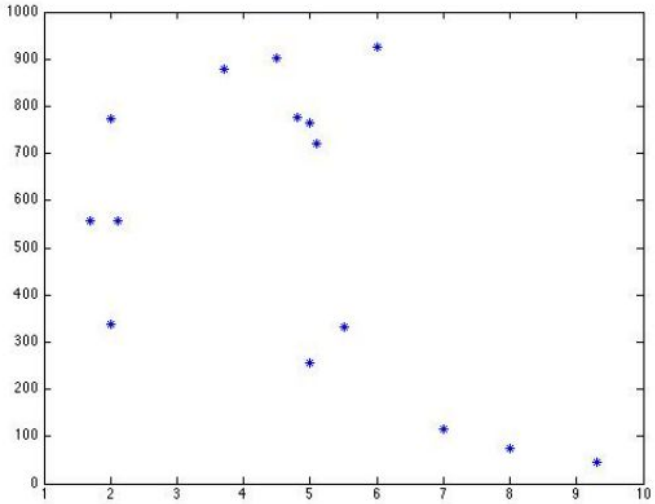
$$a' = \frac{a - \min_a}{\max_a - \min_a} (novo_\max_a - novo_\min_a) + novo_\min_a$$

Pré-processamento - Transformação dos Dados

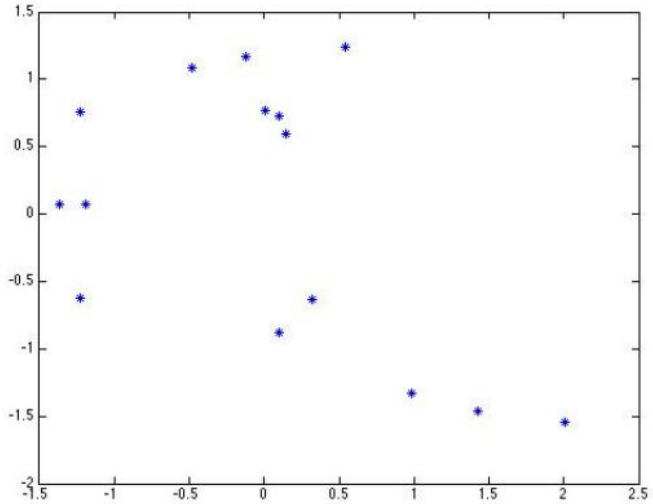
- Transformação dos dados: Normalização z-score
  - Mantém os valores centrados na média zero
  - útil para mitigar outliers
  - útil quando máximo/mínimo não são conhecidos
  - mantém a distribuição de valores em função da variância dos dados

$$a' = (a - \bar{a}) / \sigma_a$$

Pré-processamento - Transformação dos Dados

- Transformação dos dados: Normalização z-score
  - 

(w) Original



(x) Normalizado

Pré-processamento - Transformação dos Dados

- Tipos comuns de features de entrada:
  - Numéricas.
  - Binárias.
  - Evite ao máximo o uso de features categóricas!

Pré-processamento/Normalização dos Dados

- Onde consigo implementações de pré-processamento?
  - https://scikit-learn.org/stable/modules/classes.html#module-sklearn.preprocessing
- Mais usadas?

`preprocessing.Binarizer`(*[, threshold, copy])	Binarize data (set feature values to 0 or 1) according to a threshold.
`preprocessing.MaxAbsScaler`(*[, copy])	Scale each feature by its maximum absolute value.
`preprocessing.MinMaxScaler`([feature_range, ...])	Transform features by scaling each feature to a given range.
`preprocessing.Normalizer`([norm, copy])	Normalize samples individually to unit norm.
`preprocessing.OneHotEncoder`(*[, categories, ...])	Encode categorical features as a one-hot numeric array.
`preprocessing.StandardScaler`(*[, copy, ...])	Standardize features by removing the mean and scaling to unit variance

Treinando Redes Profundas

- Suponha que temos um problema muito complexo…
  - Ex.: muitas classes, tradução, detecção de objetos…
- Nesse caso, basta tratar com uma arquitetura mais complexa, certo?
  - Ex.: rede mais profunda com 10 camadas

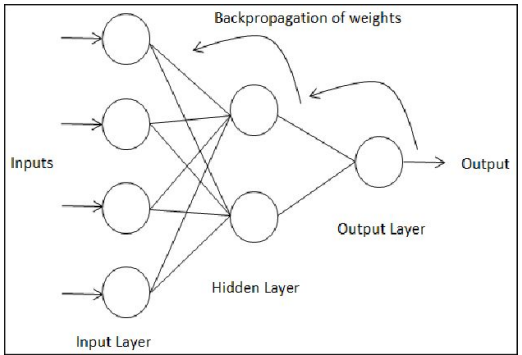
Treinando Redes Profundas

- Suponha que temos um problema muito complexo…
  - Ex.: tradução, detecção de objetos…
- Nesse caso, basta tratar com uma arquitetura mais complexa, certo?
  - Ex.: rede mais profunda com 10 camadas
- Problemas:
  - Vanish/exploding Gradients
  - Rede maior -> treino mais lento
  - Uma arquitetura complexa (milhões de parâmetros) -> Risco de overfitting
- Soluções?

Vanishing/Exploding Gradients

INSTITUTO FEDERAL
Goiás

- Como vimos, o gradiente do backpropagation é calculado a partir da camada de saída…
  - Infelizmente gradientes ficam e menores à cada camada mais próxima da entrada. Resultado:
    - Vanish gradients
    - Camadas próximas da entrada não mudam os pesos…
    - Isso atrapalha bastante o treino convergir para boas soluções
  - Eles também podem ficar cada vez maiores (veremos em redes recorrentes). Resultado:
    - Exploding gradients
    - Instabilidade no aprendizado
  - Razão pela qual redes profundas foram abandonadas por muito tempo!
  - Ou seja, redes neurais profundas sofrem de gradientes instáveis:
    - Camadas diferentes podem aprender com velocidades bastante distintas



Vanishing/Exploding Gradients

INSTITUTO FEDERAL
Goiás

Understanding the difficulty of training deep feedforward neural networks

Xavier Glorot **Yoshua Bengio**
DIRO, Université de Montréal, Montréal, Québec, Canada

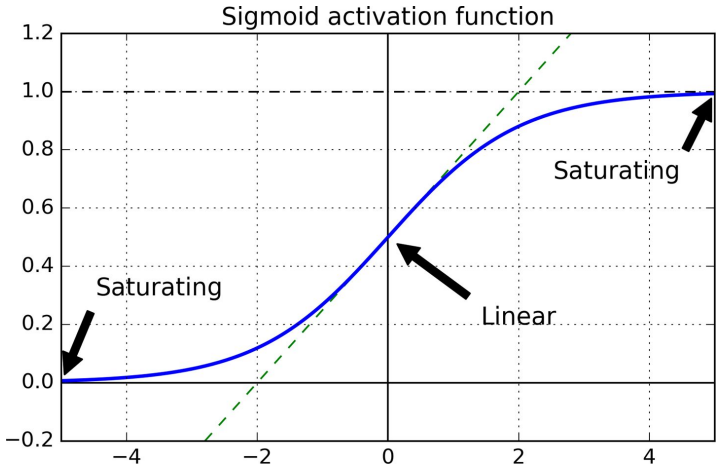
Abstract

Whereas before 2006 it appears that deep multi-layer neural networks were not successfully trained, since then several algorithms have been shown to successfully train them, with experimental results showing the superiority of deeper vs less deep architectures. All these experimental results were obtained with new initialization or training mechanisms. Our objective here is to understand better why standard gradient descent from random initialization is doing so poorly with deep neural networks, to better understand these recent relative successes and help design better algorithms in the future. We first observe the influence of the non-linear activations functions. We find that the logistic sigmoid activation

learning methods for a wide array of *deep architectures*, including neural networks with many hidden layers (Vincent et al., 2008) and graphical models with many levels of hidden variables (Hinton et al., 2006), among others (Zhu et al., 2009; Weston et al., 2008). Much attention has recently been devoted to them (see (Bengio, 2009) for a review), because of their theoretical appeal, inspiration from biology and human cognition, and because of empirical success in vision (Ranzato et al., 2007; Larochelle et al., 2007; Vincent et al., 2008) and natural language processing (NLP) (Collobert & Weston, 2008; Mnih & Hinton, 2009). Theoretical results reviewed and discussed by Bengio (2009), suggest that in order to learn the kind of complicated functions that can represent high-level abstractions (e.g. in vision, language, and other AI-level tasks), one may need *deep architectures*.

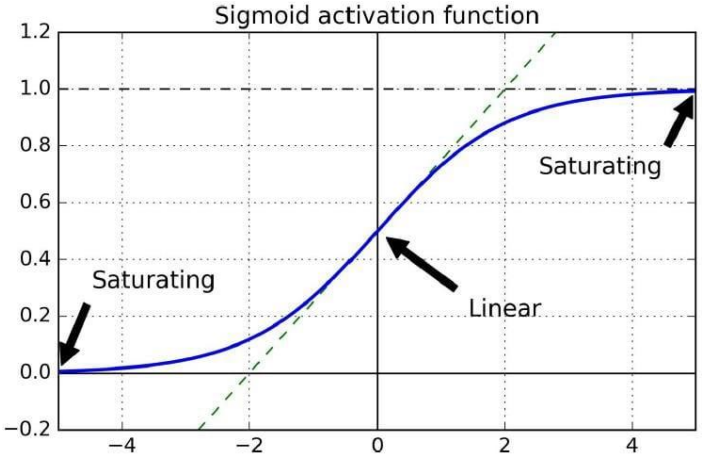
Most of the recent experimental results with deep archi-

- 2010: Xavier Glorot and Yoshua Bengio analisaram potenciais fontes do problema:
  - Inicialização dos pesos
  - Função sigmoid



Saturação - Sigmoid

- Em casos onde ocorre uma saturação, o problema de desaparecimento fica ainda pior;
- No exemplo da função de ativação logística, quando a entrada se torna grande (positiva ou negativa), a função satura em 0 ou 1, com a derivada muito próxima de 0;
- Isso piora, pois o backpropagation não terá gradiente para propagar pela rede:
  - O pouco gradiente que fica, permanece diluído nas camadas superiores, não ficando nada para as camadas baixas



Parametrização - Inicialização de Pesos

INSTITUTO FEDERAL
Goiás

- Parametrização da Inicialização:
  - Iniciar tudo com 0 ?
  - Iniciar tudo com o mesmo valor > 0 ?
  - Iniciar tudo com o mesmo valor < 0 ?
  - Iniciar com valores aleatórios com variância alta ?
  - Iniciar com valores aleatórios com variância baixa ?

Parametrização - Inicialização de Pesos

INSTITUTO FEDERAL
Goiás

- Soluções/parametrizações propostas:
  - Inicialização Glorot/Xavier (2010)
    - Para cada camada:
      - **A variância da entrada (features de entrada da camada) deve ser igual à variância dos gradientes de atualização dos pesos da saída.**
        - Padroniza o fluxo de propagação e de backpropagation
    - Distribuição uniforme entre -r e r, com:

$$r = \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$$

onde n_inputs é o número de conexões de entrada da camada, n_outputs é o número de conexões de saída da camada.

Exemplo no keras:

```
model.add(keras.layers.Dense(100,kernel_initializer="glorot_uniform"))
```

Fonte: https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf

Parametrização - Inicialização de Pesos

INSTITUTO FEDERAL
Goiás

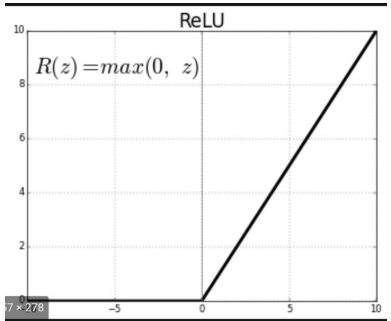
- Soluções/parametrizações propostas:
  - Inicialização Kaiming/He (2015)
    - Baseada em Glorot/Xavier
    - Adapta o cálculo da variância para função de ativação ReLU (e suas semelhantes)
      - utiliza número de conexões de entrada entrada

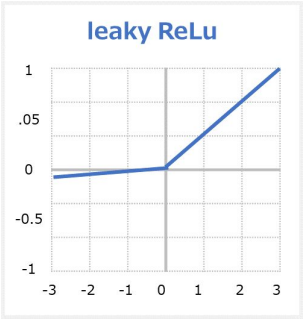
Exemplo no keras:

```
model.add(keras.layers.Dense(100,activation="relu",kernel_initializer="he_uniform"))
```

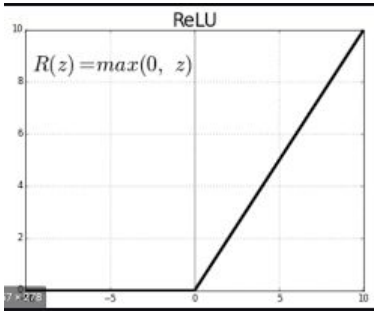
Parametrização - Inicialização de Pesos

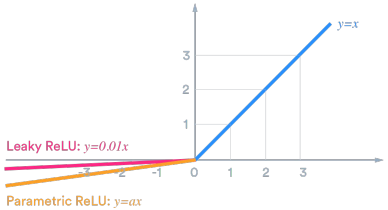
- Soluções/parametrizações propostas:
  - Funções de ativação não saturadas:
    - Segundo Glorot (2010), a escolha da função de ativação pode ser um dos culpados!
    - Até 2010, a idéia era assumir a sigmoid (neurônios biológicos)...
      - mas satura fácil no backpropagation.
    - Para redes profundas, uma idéia seria utilizar ReLU (não satura p/ positivos, e rápida de computar)
      - Alternativamente: leaky ReLU





Evitando Saturação com Funções de Ativação ReLU/Leaky ReLU

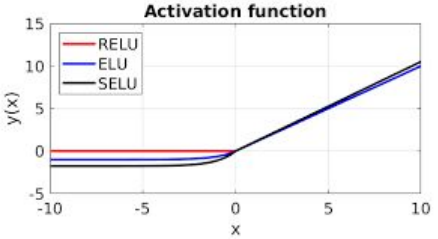




- Glorot também descobriu que a escolha da função de ativação pode melhorar muito a passada dos gradientes;
- A função de ativação, ReLU, por exemplo, não satura para valores positivos.
- Contudo, a ReLU tem o problema dying ReLU:
  - Durante o treino alguns neurônios simplesmente morrem, ou seja, a saída do neurônio é somente zero;
  - Ou seja, os pesos de um neurônio ficam ajustados de tal forma que a soma dos pesos da entrada são negativas para todas as instâncias. Assim a saída da função fica em zero;
  - Logo, GD não afeta em mais nada, pois o gradiente da ReLU é zero quando a função é negativa
- Uma alternativa para isso é a “Leaky ReLU”, LeakyReLU(z)= max(αz, z)

Evitando Saturação com Funções de Ativação: Scaled ELU (SELU)

- Proposta recente (2017), por Guter Klambauer
- O autor mostra que se a rede for composta exclusivamente de pilhas de camadas densas, e todas as camadas ocultas usarem SELU como função de ativação, então a rede irá se auto-organizar:
  - A saída de cada camada tende a preservar média de zero e desvio padrão de 1 durante o treino;
  - Mitiga o problema de desaparecimento do gradiente.
- Para que o resultado seja bom, a SELU exige:
  - Kernel_initializer =“lecun_normal”
  - A rede precisa ser sequencial, logo não vale para redes recorrentes



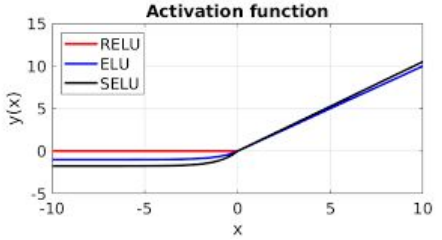
$$\mathrm{selu}(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leqslant 0 \end{cases}.$$

Exemplo no keras:

```
model.add(keras.layer.Dense(10, activation=“selu”, kernel_initializer=“lecun_normal”))
```

Evitando Saturação com Funções de Ativação: Scaled ELU (SELU)

- Porque pouca gente usa SELU, ELU e outras funções da família RELU na prática?
  - Relu é *muito* barata de computar.
  - Relu se tornou um padrão nas arquiteturas. Trocar a função significa repensar uma arquitetura.
  - É difícil comparar o seu modelo com outros modelos que já usam RELU.
    - Em outras palavras, para aplicar a um novo problema…
      - use o que todo mundo usa!
      - tente manter, sempre que possível, padrões dos criadores da arquitetura.
        - … mas saiba da existência de outras opções de parametrização.



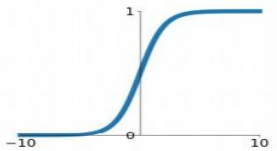
$$\text{selu}(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leqslant 0 \end{cases}.$$

Escolhas de Parametrização - Ativação

- Hyperparâmetros
  - Função de Ativação mais usadas

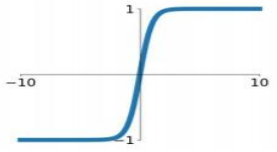
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



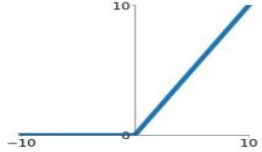
tanh

$$\tanh(x)$$



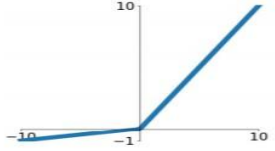
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

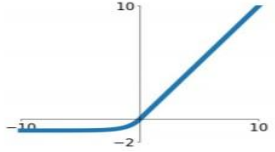


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

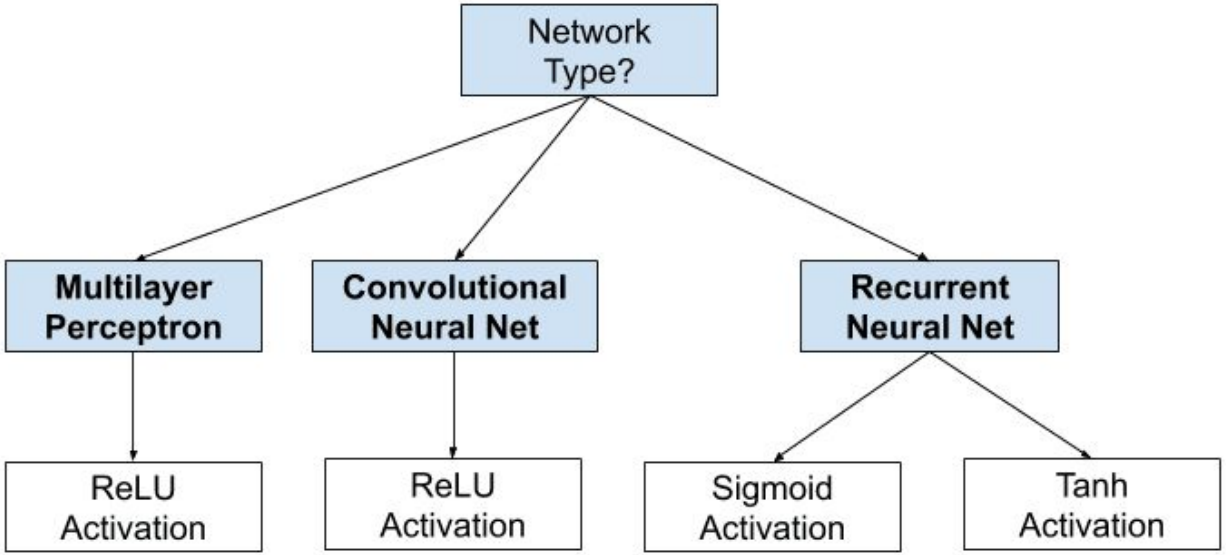


Escolhas de Parametrização - Ativação

- Hiperparâmetros: Função de Ativação
  - *"… the hyperbolic tangent activation function typically performs better than the logistic sigmoid." Ian Goodfellow- Deep Learning.*
    - Entretanto, tanto tanh quanto sigmoid saturam, e sua utilização depende da arquitetura e outras escolhas de parametrização. Arquiteturas de redes recorrentes, LSTM, GRU dependem dessas funções.
  - *"In modern neural networks, the default recommendation is to use the rectified linear unit or ReLU …"*

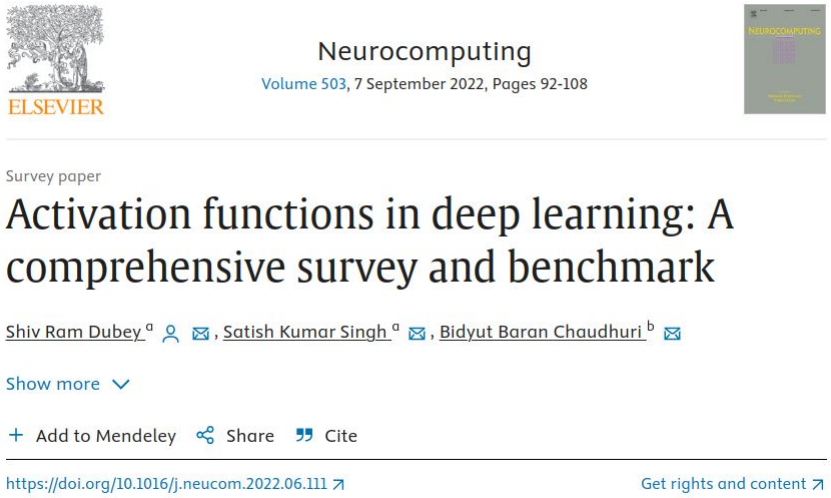
Escolhas de Parametrização - Ativação

- Hiperparâmetros: Função de Ativação
  - Portanto, segundo ***Ian***, para as camadas ocultas:



Escolhas de Parametrização - Ativação

- Hyperparâmetros
  - Quais são as melhores Funções de ativação?
    - Difícil…: https://doi.org/10.1016/j.neucom.2022.06.111
    - https://ml-explained.com/blog/activation-functions-explained



ELSEVIER

Neurocomputing

Volume 503, 7 September 2022, Pages 92-108

Survey paper

Activation functions in deep learning: A comprehensive survey and benchmark

Shiv Ram Dubey [a], Satish Kumar Singh [a], Bidyut Baran Chaudhuri [b]

Show more

+ Add to Mendeley Share Cite

https://doi.org/10.1016/j.neucom.2022.06.111

Get rights and content

Escolhas de Parametrização - Ativação

INSTITUTO FEDERAL
Goiás

- Hyperparâmetros

Table 8. Experimental results comparison over CIFAR10 dataset.

Accuracy	CNN Models					
Activations	**MobileNet**	**VGG16**	**GoogleNet**	**ResNet50**	**SENet18**	**DenseNet121**
Sigmoid	88.60 ± 0.17	87.69 ± 0.49	87.33 ± 2.48	80.13 ± 3.33	90.29 ± 0.29	89.92 ± 1.96
Tanh	87.21 ± 0.24	90.49 ± 0.11	90.16 ± 1.86	89.09 ± 1.47	90.44 ± 0.09	91.80 ± 0.69
ReLU [8]	90.10 ± 0.22	*92.84* ± 0.19	*93.43* ± 0.48	93.74 ± 0.34	93.70 ± 0.16	**93.96** ± 0.51
LReLU [17]	90.10 ± 0.19	91.09 ± 0.09	89.28 ± 0.82	*93.83* ± 0.42	93.66 ± 0.19	93.85 ± 0.48
ELU [27]	90.92 ± 0.25	88.55 ± 1.17	92.47 ± 0.76	93.53 ± 0.66	93.39 ± 0.20	92.89 ± 0.62
SELU [52]	90.11 ± 0.32	92.25 ± 0.28	91.87 ± 0.84	93.53 ± 0.52	89.96 ± 0.31	92.71 ± 0.73
GELU [101]	90.71 ± 0.20	92.42 ± 0.09	93.16 ± 0.61	93.81 ± 0.46	**93.72** ± 0.18	*93.90* ± 0.41
CELU [53]	*91.04* ± 0.17	88.11 ± 0.14	92.60 ± 0.60	**94.09** ± 0.17	91.63 ± 0.22	93.46 ± 0.35

Escolhas de Parametrização - Ativação

- Hyperparâmetros

Fig. 3. Convergence plots over CIFAR100 dataset.

Table 10. Training time (hh:mm:ss) comparison over CIFAR100 dataset.

Training Time	CNN Models					
Activations	**MobileNet**	**VGG16**	**GoogleNet**	**ResNet50**	**SENet18**	**DenseNet121**
Sigmoid	00:33:15	00:49:16	04:55:54	03:36:03	01:13:14	04:12:24
Tanh	00:33:18	00:49:55	04:58:02	03:33:03	01:13:18	04:09:24
Elliott [25]	00:49:52	00:59:13	06:53:55	05:38:49	01:41:38	07:46:55
ReLU [8]	00:31:22	00:47:19	04:55:10	03:32:30	01:15:33	04:15:06
LReLU [34]	00:31:48	00:49:03	05:01:30	03:33:00	01:18:38	04:14:09
PReLU [35]	00:44:24	00:49:01	05:42:18	03:55:57	01:27:05	04:55:47
ELU [27]	00:31:05	00:47:38	04:57:37	03:36:47	01:13:25	04:08:39
SELU [52]	00:29:40	00:47:31	04:54:57	03:33:47	01:13:27	04:09:17
GELU [101]	00:29:43	00:47:22	04:55:53	03:32:32	01:13:32	04:11:26

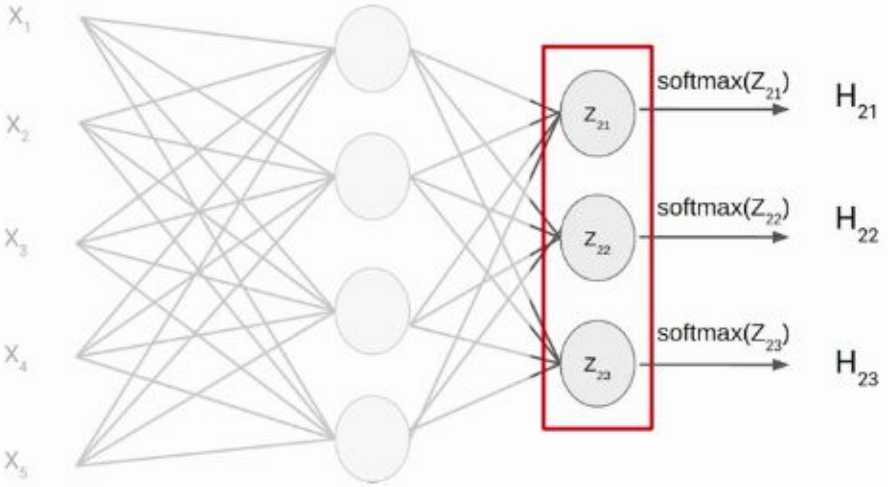
Escolhas de Parametrização - Ativação

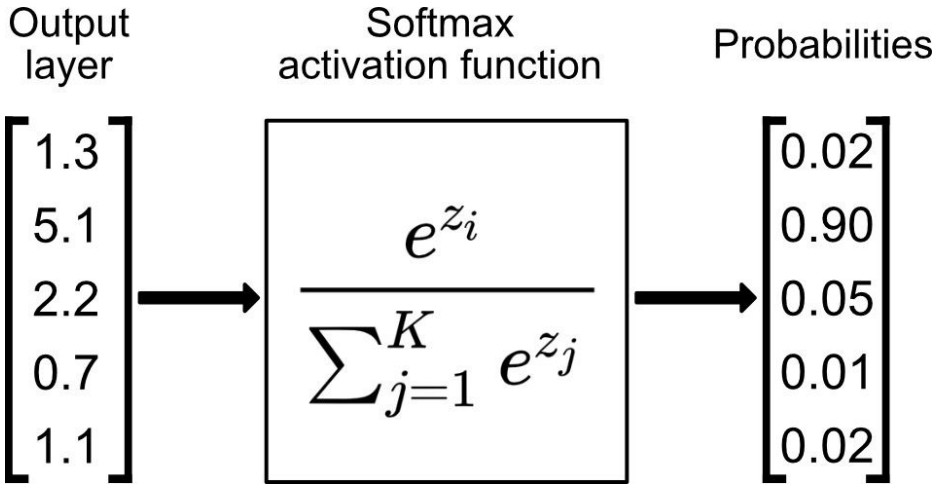
INSTITUTO FEDERAL
Goiás

- Hiperparâmetros: Função de Ativação típicas da camada de saída
  - Linear `activation="linear"`
    - Saída sem limite superior/inferior. Usualmente p/ regressão.
  - Sigmoid - `activation="sigmoid"`
    - classificação/multilabel classification
  - Softmax - `activation="softmax"`
    - “Força” uma saída (multiclass classification)
    - Múltiplos neurônios de saída e um deve ser escolhido.

Escolhas de Parametrização - Ativação

- Hiperparâmetros: Função de Ativação da camada de saída
  - `keras.layers.Dense(3,activation="softmax")`





Parametrização - Batch Normalization

- *Problema:*
  - *Após a inicialização, os pesos das diferentes camadas da rede podem desviar muito uns dos outros durante o treino…*
    - *Ex.: para cada batch de entrada, se os dados dos diferentes batches são muito diferentes (variação muito alta), o ajuste dos pesos se torna uma tarefa de "acertar um alvo em movimento"*
- Em 2015, Sergey Ioffe e Christian Szegedy propuseram Batch Normalization (BN)
  - *"Training Deep Neural Networks is complicated by the fact that the distribution of each layer's inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models" (Sergey Ioffe, 2015)*
- A estratégia é adicionar uma operação antes ou depois de cada camada oculta;
- Para cada batch, a operação irá normalizar a entrada e ajustar a variação para média em zero;
- Basicamente a operação deixa o modelo aprender a melhor escala e média de cada camada de entrada

Limitações do BatchNorm e a Solução LayerNorm

INSTITUTO FEDERAL

O Problema do Batch Normalization (BN)

O BatchNorm depende crucialmente de estatísticas calculadas ao longo do **mini-batch**. Isso gera sérios problemas em cenários específicos:

1. Batches Pequenos ou Dinâmicos

Se o tamanho do lote for muito pequeno (ex: treinamento com poucos recursos), as estimativas de média e variância tornam-se ruidosas e instáveis, degradando o desempenho.

2. Dados de Tamanho Variável (RNNs)

Em redes recorrentes, o comprimento das sequências muda. Manter estatísticas separadas por passo de tempo em lotes variados torna-se extremamente complexo.

3. Discrepância Treino vs. Inferência

Exige guardar médias móveis durante o treino para usar na inferência, adicionando complexidade operacional.

Por que o Layer Normalization (LN) Supera?

Proposto por Jimmy Lei Ba, Jamie Ryan Kiros e **Geoffrey E. Hinton** em 2016 como uma alternativa robusta ao Batch Normalization.Resolve os gargalos do BN ao normalizar as ativações **através dos neurônios (features)** de cada instância isoladamente.

Vantagens Cruciais do LayerNorm:

- **Independência Total do Batch:** O cálculo é idêntico no treino e na inferência, eliminando oscilações por tamanho de lote.
- **Perfeito para NLP e LLMs:** Essencial para Transformers e RNNs, pois processa cada sentença de forma autocontida, mesmo com comprimentos variáveis.
- **Estabilidade e Isolamento:** Evita o problema de "alvo em movimento" ao não misturar informações entre diferentes exemplos do mesmo lote.

Keras API:

```
keras.layers.LayerNormalization()
```

LayerNormalization - Formulação e Visualização

É essencialmente o Z-SCORE das features

O z-score clássico é:

$$z_i = (x_i - \mu) / \sigma$$

A LayerNorm faz praticamente isso:

$$\hat{x}_i = (x_i - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$$

onde ε é um valor pequeno para evitar divisão por zero.



Implementando Batch Normalization

```
In [35]: model = keras.models.Sequential([
             keras.layers.Flatten(input_shape=[28, 28]),
             keras.layers.BatchNormalization(),
             keras.layers.Dense(300, activation="relu"),
             keras.layers.BatchNormalization(),
             keras.layers.Dense(100, activation="relu"),
             keras.layers.BatchNormalization(),
             keras.layers.Dense(10, activation="softmax")
         ])
```

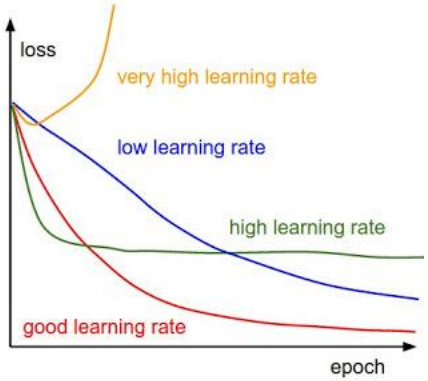
```
In [36]: model.summary()
Model: "sequential_4"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten_4 (Flatten)          (None, 784)               0
_________________________________________________________________
batch_normalization (BatchNo (None, 784)               3136
_________________________________________________________________
dense_212 (Dense)            (None, 300)               235500
_________________________________________________________________
batch_normalization_1 (Batch (None, 300)               1200
_________________________________________________________________
dense_213 (Dense)            (None, 100)               30100
_________________________________________________________________
batch_normalization_2 (Batch (None, 100)               400
_________________________________________________________________
dense_214 (Dense)            (None, 10)                1
=================================================================
Total params: 271,346
Trainable params: 268,978
Non-trainable params: 2,368
```

BN adiciona 4 parâmetros por entrada: variâncias e médias e dois parâmetros para ajustar a média e variancia de acordo com a entrada. Como nesse caso são 784 entradas: 784x4=3136

Veja que agora nós temos parâmetros não treináveis. Isso pois a média e variância do BN não são ajustados pelo back-propagation, então não são treináveis.
Total de parâmetros BN
(3136+1200+400)/2 = 2368

Taxa de Aprendizado (Learning Rate, LR)

- Como já discutido:
  - Taxa de aprendizado alta, mostra uma alta confiança no cálculo do erro e a convergência deve ficar dançando em relação ao ponto mínimo;
  - Taxa de aprendizado baixa, o treino deverá chegar ao valor ótimo, mas levará muito tempo.
- Quando se tem pouco recurso para experimentos, provavelmente o experimento deverá ser interrompido sem ter chegado em uma boa convergência



LR Schedules ou Dinâmico

- Começando com valores maiores e ir diminuindo à medida que o treino melhora;

Power Scheduling: $\eta(t) = \eta_0/(1 + \frac{t}{s})$

- Sendo t o número de iteração, ou epoch, e η_0 e s hiper-parâmetros
- Veja que LR diminui a cada nova iteração

Exponencial Scheduling: $\eta(t) = \eta_0\ 0.1^{t/s}$

- Diferente de Power Scheduling, a redução aqui é mais drásticas, por fator de 10 a cada passo

Piecewise Constant Scheduling

- Definir uma valor de LR para cada intervalo de epochs
- Exemplo: η_0 =0.1 para 10 epochs iniciais, 0.2 para outras 20....
- Conjunto de validação poderá ajudar.

LR Schedules - Implementando

- Power Scheduling: $\eta(t) = \eta_0/(1 + \frac{t}{s})$
  - Basta usar hiperparâmetro decay para parâmetro s. Embora decay seja o inverso de s da função.
  - Optimizer= keras.optimizers.SGD(lr=0.01, decay=1e-4)

Exponencial Scheduling: $\eta(t) = \eta_0\ 0.1^{t/s}$
- Criar uma função:
  - def exponential_decay_fn(epoch):
    - Return 0.01*0.1**(epoch/20)
- Agora crie LearningRateScheduler call-back
  - lr_scheduler= keras.call-backs.LearningRateScheduler(exponential_decay_fn)
  - History = model.fit(X_train_scaled, y_train, [...], callbacks= [lr_scheduler])

Piecewise Constant Scheduling
- Basta mudar a função exponential_decay_fn para:
  - def piecewise_constant_fn(epoch):
    - If epoch < 5:
      - return 0.01
    - elif epoch < 15:
      - return 0.005
    - else
      - return 0.001

Early Stop - Parando o treinamento

- Idéia: Quando paramos de aprender (não há evolução na loss), podemos parar o treinamento.
  - Mais usado!
- O modelo avalia se após ***P*** interações (patience), o erro de validação não é reduzido. Nesse caso, ele pára o treinamento.
  - `es = EarlyStopping(monitor='val_loss', mode='min', patience=5)` ou
  - `es = EarlyStopping(monitor='val_accuracy', mode='max',patience=5)`

- Executado com uma coleção de validação:
  - `model.fit(train_X, train_y, validation_data=(val_x, val_y), callbacks=[es])` ou
  - `model.fit(train_X, train_y, validation_split=0.3, callbacks=[es])`

INSTITUTO FEDERAL
Goiás

Deep Learning

Lidando com (Hiper)parâmetros

Regularização

Evitando Overfitting por Regularizaton

- *With four parameters I can fit an elephant and with five I can make him wiggle his trunk*. (John von Neuman.)
- Com centenas de parâmetros, você consegue modelar todo o zoológico!
- Redes neurais usam centenas e até milhares de parâmetros, gerando uma grande liberdade para ajustar uma grande variedade de dados complexos;
- A grande questão é adaptar o “sobre ajuste” desses parâmetros, ou overfitting.
  - Quem ainda não viu, veja: https://www.youtube.com/watch?v=DQWI1kvmwRg
- Para tentar mitigar esse problema, temos:
  - L1 e L2 regularization
  - Dropout
  - Max-Norm regularization

L1 e L2 Regularization

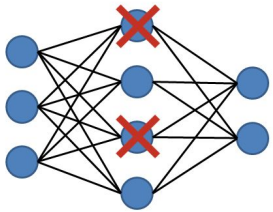
- Quanto menos graus de liberdade (variações) os pesos podem ter mais difícil será cair no overfitting:
  - Idéia: "minimizar" o módulo do valor dos pesos diretamente na função de custo.
  - L1 - efeito de seleção de atributos (mais agressivo)
  - L2 - Efeito de redução de importância de atributos
  - Valores razoáveis variam entre 0 e 0.1 (Applied Predictive Modeling)

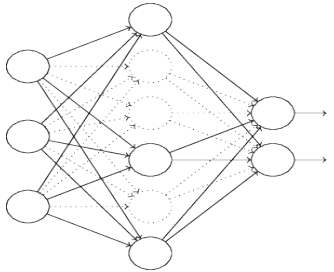
In [101]:
```
layer = keras.layers.Dense(100, activation="elu",
                           kernel_initializer="he_normal",
                           kernel_regularizer=keras.regularizers.l2(0.01))
# or l1(0.1) for ℓ1 regularization with a factor or 0.1
# or l1_l2(0.1, 0.01) for both ℓ1 and ℓ2 regularization, with factors 0.1 and 0.01 respectively
```

In [102]:
```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="elu",
                       kernel_initializer="he_normal",
                       kernel_regularizer=keras.regularizers.l2(0.01)),
    keras.layers.Dense(100, activation="elu",
                       kernel_initializer="he_normal",
                       kernel_regularizer=keras.regularizers.l2(0.01)),
    keras.layers.Dense(10, activation="softmax",
                       kernel_regularizer=keras.regularizers.l2(0.01))
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam", metrics=["accuracy"])
n_epochs = 2
history = model.fit(X_train_scaled, y_train, epochs=n_epochs,
                    validation_data=(X_valid_scaled, y_valid))
```

$$\hat{\mathcal{L}}(W) = \frac{\alpha}{2}||W||_2^2 + \mathcal{L}(W) = \frac{\alpha}{2}\sum_i\sum_j w_{ij}^2 + \mathcal{L}(W)$$

Dropout





- Proposta de regularização feita em 2012, que mostra ganhos de 1% a 2% nos modelos de forma geral: algo útil quando se está com 95% de acurácia.
- O algoritmo é: para cada passo de ajuste dos pesos no treinamento, o neurônio terá probabilidade ***p*** de temporariamente não participar do treinamento, não o incluindo como neurônio de saída.
- Hiperparâmetro: p ou dropout rate.
  - Geralmente algo entre 10% e 50%
- A ideia é fazer com que os neurônios não dependam excessivamente de outros neurônios;
- Os neurônios precisam se adaptar com a ausência de outros;
- Isso contribui a ser menos sensível a mudanças no treino.
- Uma outra intuição: com dropout cada epoch treina uma rede diferente, e todas se juntariam para dar o resultado.

Implementando Dropout

- Tome cuidado ao avaliar training e validation loss, pois a análise ocorre com a ausência de alguns neurônios, algo que não irá ocorrer no teste.
- Se o overfitting aumentar, devemos aumentar dropout

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(10, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam", metrics=["accuracy"])
n_epochs = 2
history = model.fit(X_train_scaled, y_train, epochs=n_epochs,
                    validation_data=(X_valid_scaled, y_valid))
```

```
Train on 55000 samples, validate on 5000 samples
Epoch 1/2
55000/55000 [==============================] - 8s 139us/sample - loss: 0.5856 - accuracy: 0.7992 - val_loss: 0.3908 - val_accuracy: 0.857
0
Epoch 2/2
55000/55000 [==============================] - 6s 117us/sample - loss: 0.4260 - accuracy: 0.8443 - val_loss: 0.3389 - val_accuracy: 0.873
0
```

Max-Norm Regularization

- A ideia: para cada neurônio, o método restringe os pesos w da conexão de entrada tal que:
  - $||w||_2 \leq r$
  - Tal que r é um hiperparâmetro e $||.||_2$ é L2 norm

Diferente de uma regularização geral para Loss Function, Max-norm aplica escala para os pesos de forma que: w← w/ $||w||_2$

Implementando:

```
layer = keras.layers.Dense(100, activation="selu", kernel_initializer="lecun_normal",
                           kernel_constraint=keras.constraints.max_norm(1.))
```

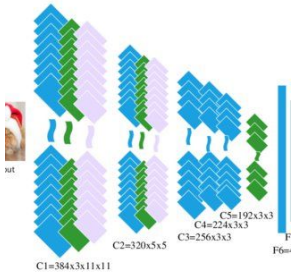
Fine-Tuning: Número de Camadas Ocultas

- Um MLP com uma camada oculta pode teoricamente aprender diversos padrões;
- Contudo, para problemas complexos uma ampla avaliação dos hiperparâmetros é algo necessário.
- Camadas baixas modelam estruturas de mais baixo nível:
  - Formas de variação e orientação linear
- Camadas intermediárias combinam estas para estruturas intermediárias
  - Formas de variação quadráticas ou lineares
- Camadas avançadas e de saída combina as estruturas anteriores para estruturas mais avançadas
  - Faces de pessoas

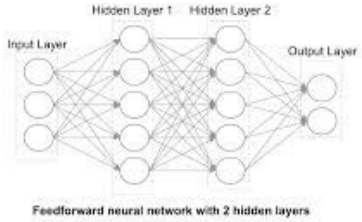


Fine-Tuning: Número de Neurônios por Camada Oculta

- Modelo Pirâmide

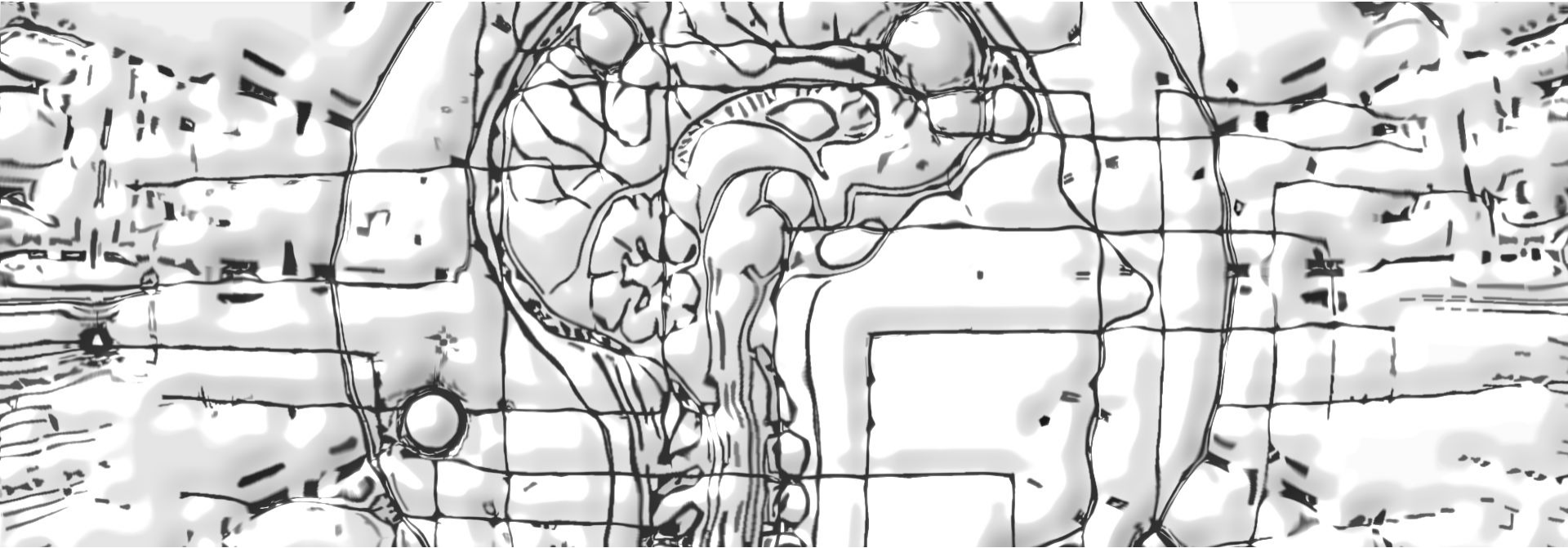


- Mesma quantidade de neurônios

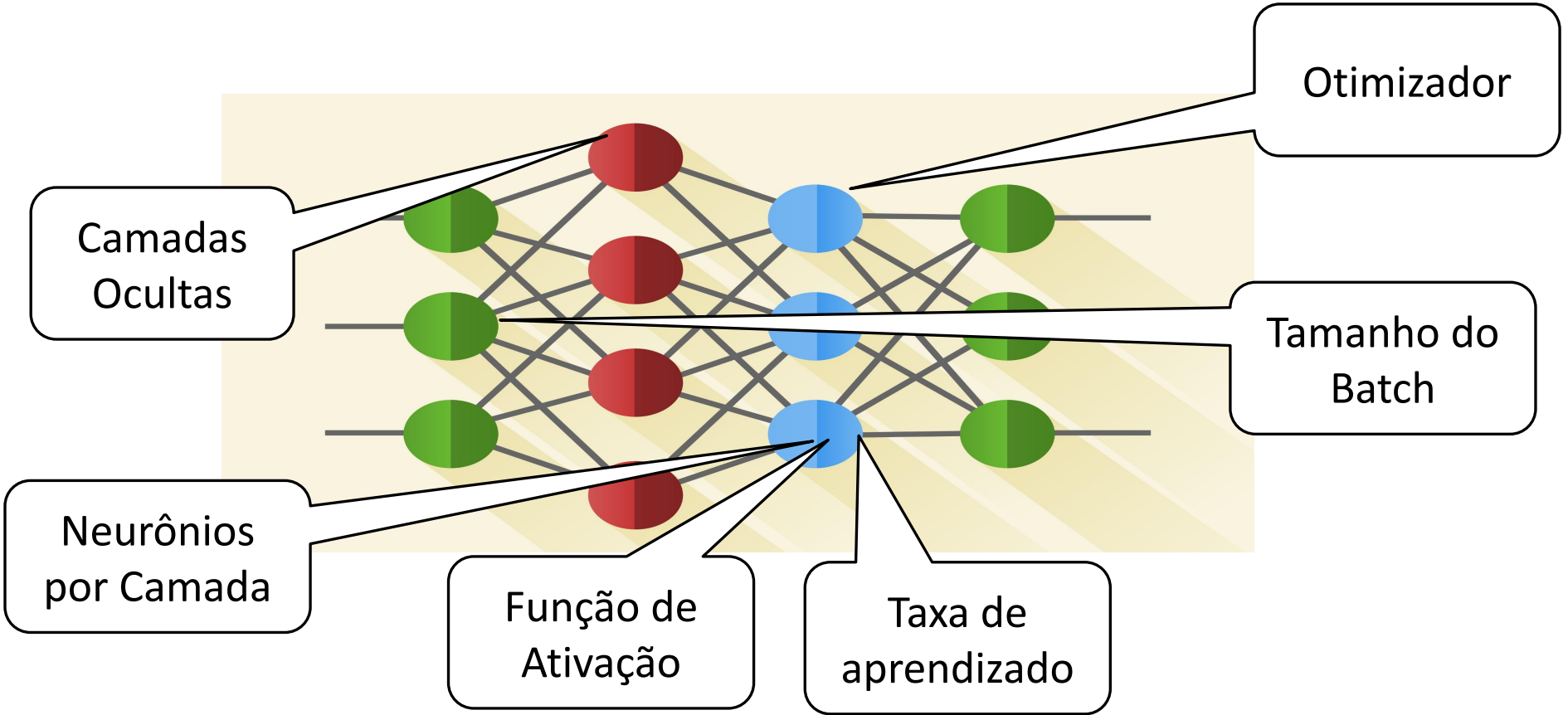


Feedforward neural network with 2 hidden layers

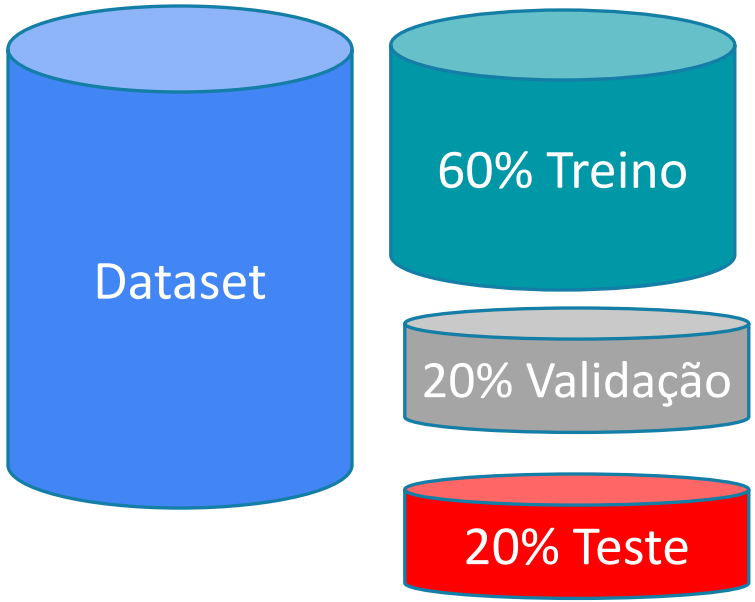
Fine-Tuning NN Hyperparameters



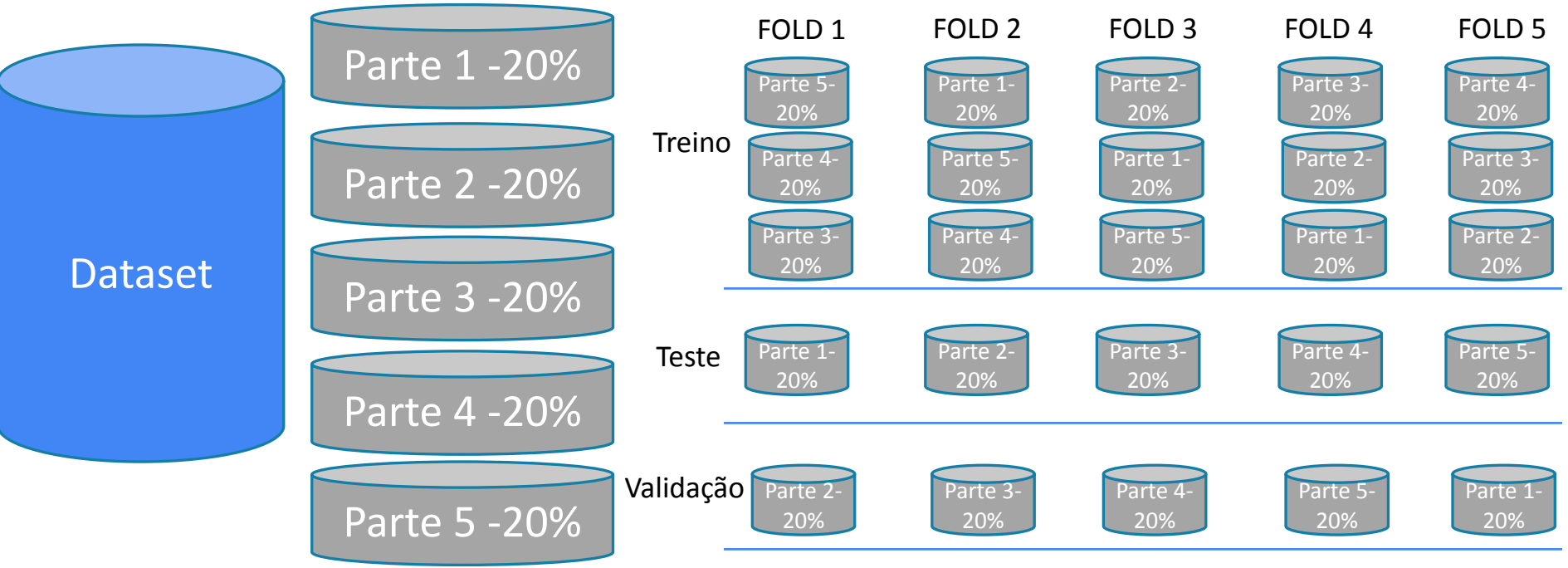
Fine-Tuning NN Hyperparameters



Fine-Tuning: Teste



Fine-Tuning: 5-Fold CrossValidation

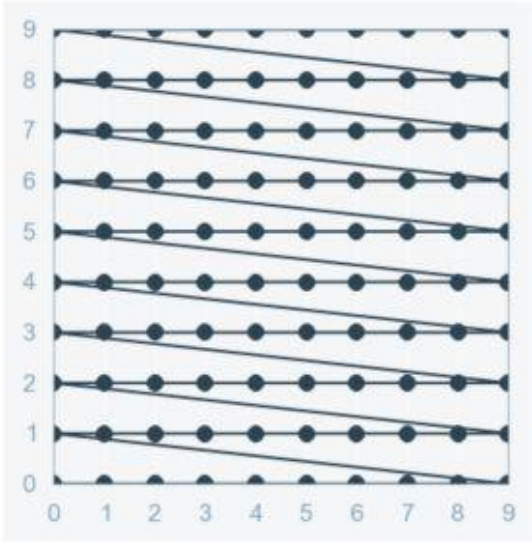


Importância do Cross-Validation

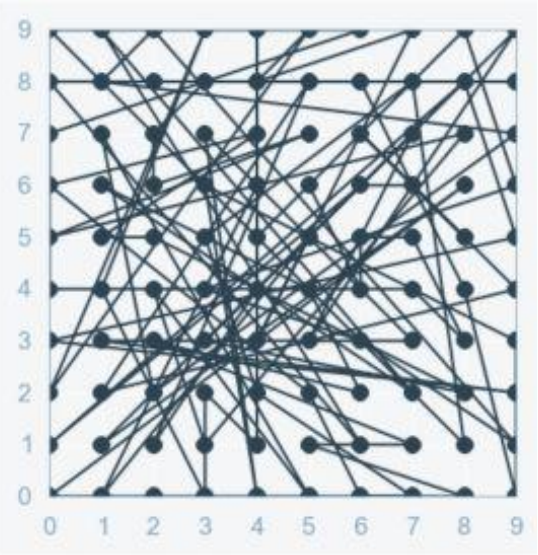
- Execução de múltiplos experimentos com um mesmo conjunto de dados.
- Viabiliza análise de dispersão dos resultados
  - … com execuções treinamento distintas.
  - … testados em partições diferentes do conjunto de dados.
- Evita o "azar" ou "sorte" de escolher um conjunto de teste fácil/difícil.
- Viabiliza análises estatísticas sobre as métricas utilizadas para obtenção dos resultados.
  - Intervalo de confiança.
  - Testes pareados.

Grid Search vs Random Search vs Bayesian Search

```
param_grid = [
    'n_estimators': [3, 10, 30],
    'max_features': [2, 4, 6, 8],
    'bootstrap' : [True, False]
]
```



Visual Representation of grid search



Visual Representation of Random search

Para dois parâmetros, variando de 0 a 9:

Entendendo a Busca Bayesiana (Bayesian Search)

Estratégia inteligente baseada em probabilidade para otimização de hiperparâmetros

Como Funciona?

Diferente do Grid ou Random Search, a Busca Bayesiana analisa os resultados de testes anteriores para decidir de forma lógica qual será a próxima tentativa.

- Cria um **Modelo Substituto** (ex: Processo Gaussiano) para mapear os hiperparâmetros.
- Usa uma **Função de Aquisição** para balancear exploração e aproveitamento.

Vantagens

- **Maior Eficiência:** Busca os melhores parâmetros com muito menos interações e tempo computacional.
- **Ideal para Modelos Pesados:** Excelente quando o treino de cada modelo é extremamente demorado ou custoso.
- **Decisões Inteligentes:** Evita testar regiões do espaço de busca que sabidamente produzem métricas ruins.

Otimização de Hiperparâmetros com Optuna

Parametrização moderna e eficiente baseada em algoritmos Bayesianos (TPE)

```python
import optuna
import keras

def objective(trial):
    # Definir o espaço de busca de hiperparâmetros
    n_layers = trial.suggest_int("n_layers", 1, 3)
    lr = trial.suggest_float("lr", 1e-4, 1e-2, log=True)
    #se for parâmetro com limite superior/inferior:
    neurons = trial.suggest_int("neurons", 10, 100)

    #se for parâmetro discreto:
    #neurons = trial.suggest_categorical("neurons", [10, 20, 30])

    model = create_model(n_layers, neurons, lr)
    history = model.fit(X_train, y_train, epochs=10,
                        validation_data=(X_val, y_val), verbose=0)
    val_loss = history.history["val_loss"][-1]

    return val_loss

# Criar o estudo e executar a otimização
study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=30)
```

Vantagens do Optuna

- Busca Bayesiana (TPE) mais inteligente e rápida que o Random ou Grid Search.

Espaço de Busca Dinâmico

- Permite condicionar parâmetros (ex: definir quantidade de neurônios por camada de forma dinâmica usando loops Python nativos).

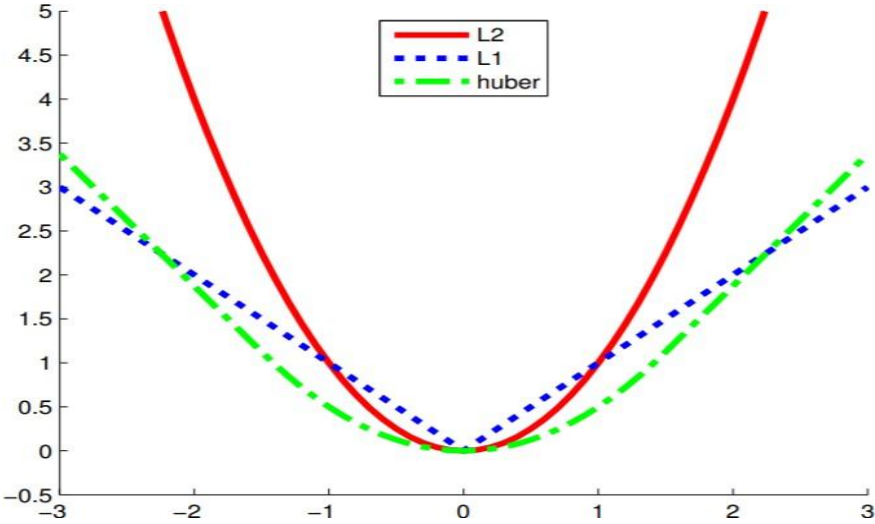
Documentação e Exemplos

Acesse a API oficial e tutoriais de integração:

https://optuna.org/

Definição da Função Custo:

- Regressão



As mais usadas são:

MSE loss (L2 loss) $\mathbb{E}_{\mathbf{x},\mathbf{y}\sim p_{\text{data}}} ||\boldsymbol{y} - f(\boldsymbol{x})||^2$

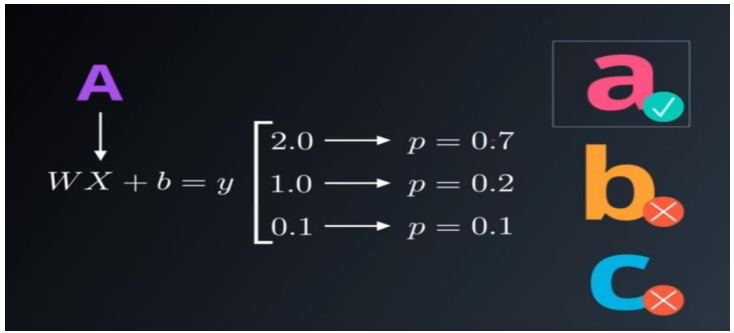
```
MSE_loss = mean(square(y_true - y_pred))
```

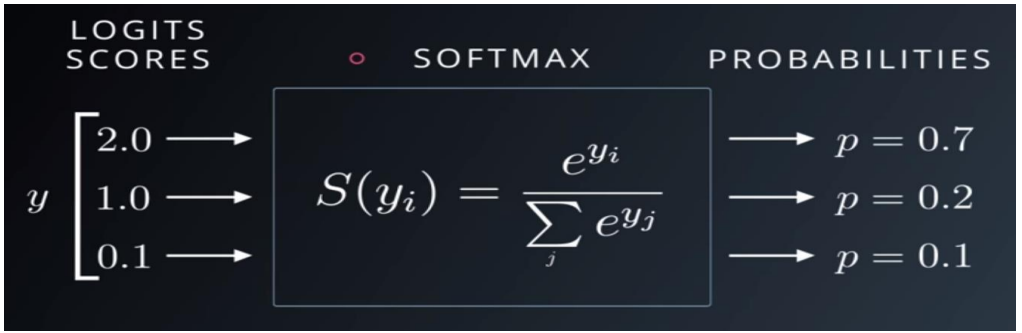
MAE loss (L1 loss) $\mathbb{E}_{\mathbf{x},\mathbf{y}\sim p_{\text{data}}} ||\boldsymbol{y} - f(\boldsymbol{x})||_1$

```
MAE_loss = mean(abs(y_true - y_pred))
```

Definição da Função Custo:

- Classificação





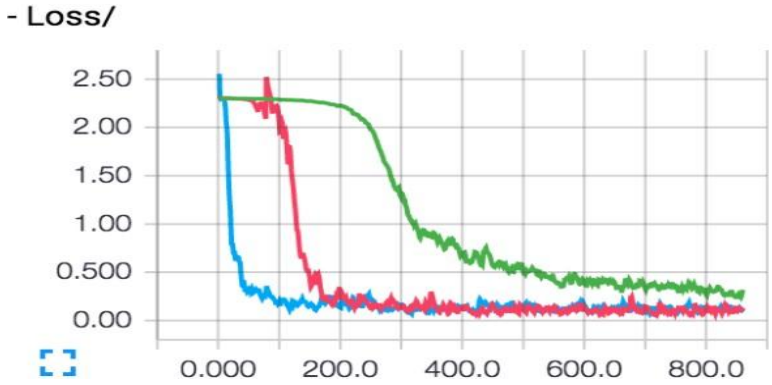
Entropia Cruzada:

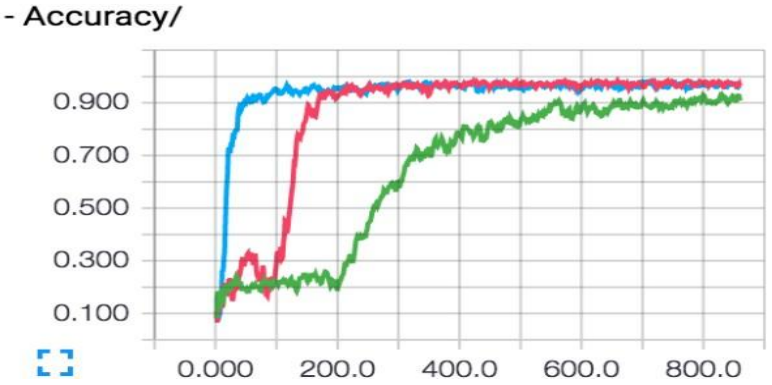
$$C = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\ln(\hat{y}_{ik})$$

Outros exemplos menos usados: Hinge, divergência de Kullback-Leibler, similaridade cosseno, etc. https://keras.io/api/losses/)

Definição da Métrica:

- Diferença entre Custo e Métrica

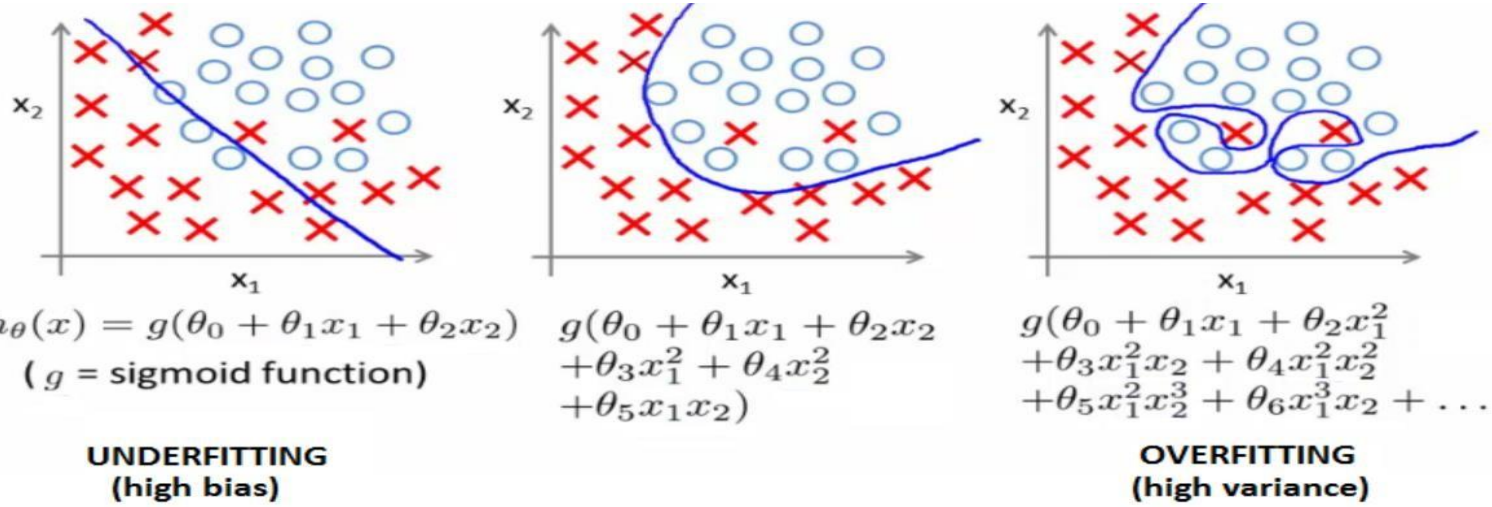




- Exemplos de métricas: acurácia, precisão, recall, F1 score, MAP, etc.

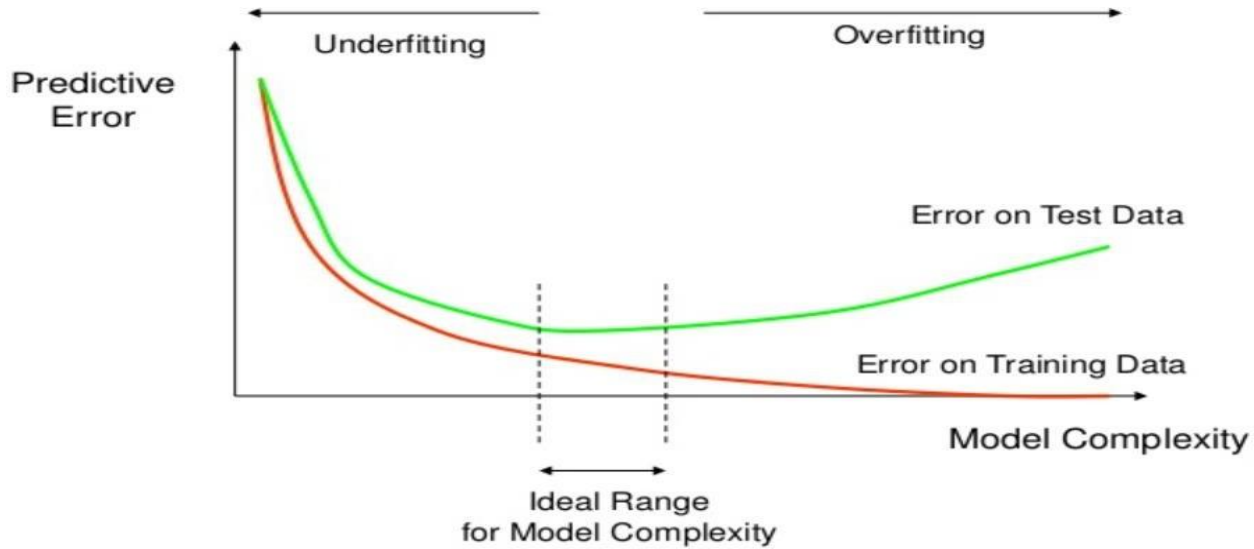
Interpretação e Correção de Resultados:

- Underfit vs Overfit



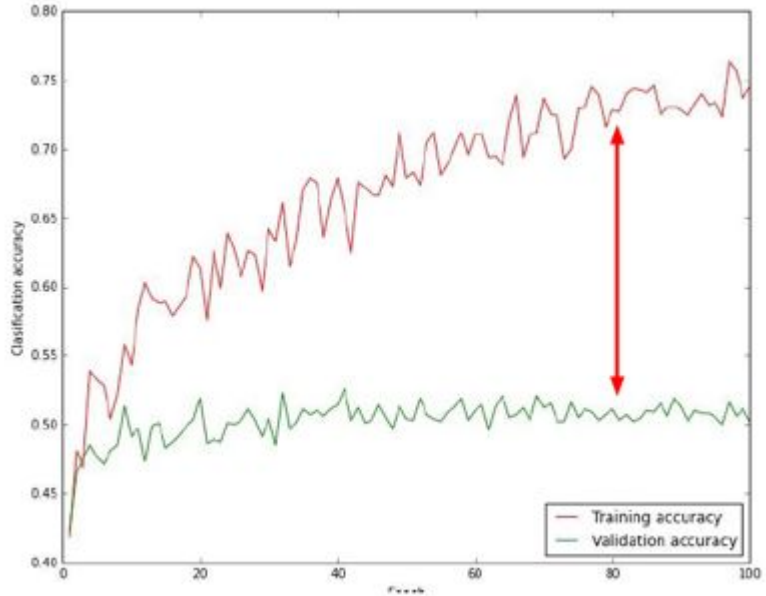
Interpretação e Correção de Resultados:

- Underfit vs Overfit



Interpretação e Correção de Resultados:

- Detectando Overfitting



Interpretação e Correção de Resultados:

- Detectando Overfitting

Training Error	Dev Error
15%	30%

We estimate the bias as 15%, and variance as 15%. This classifier has **high bias and high variance**: It is doing poorly on the training set, and therefore has high bias, and its performance on the dev set is even worse, so it also has high variance. The overfitting/underfitting terminology is hard to apply here since the classifier **is simultaneously overfitting and underfitting**.

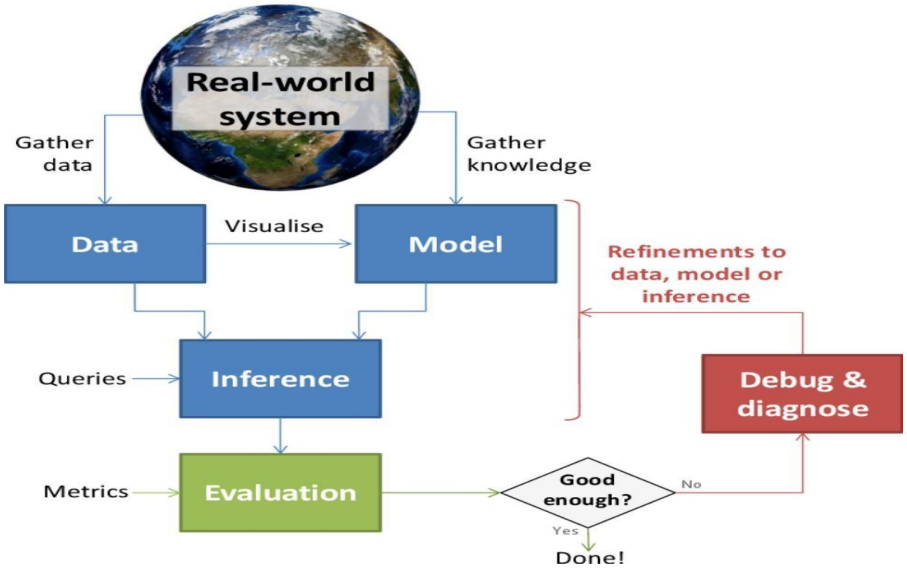
Fonte: Machine Learning Yearning (Andrew NG)

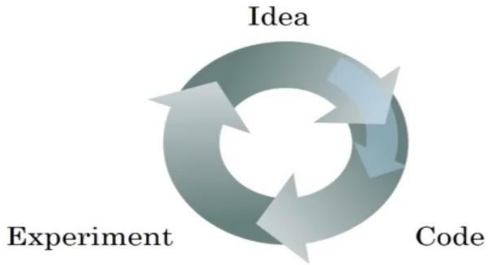
Interpretação e Correção de Resultados:

- Parâmetros: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, W^{[3]}, b^{[3]} \ldots$
- Hyperparâmetros:
  - Arquitetura da rede (MLP, CNN, RNN, …)
  - Número de camadas e número de unidades por camada (profundidade e largura da rede)
  - Função de ativação
  - Algoritmos de Otimização
  - Taxa de aprendizado
  - Técnicas de regularização
  - […] (sempre tem um que vc tá esquecendo... --')

Interpretação e Correção de Resultados:

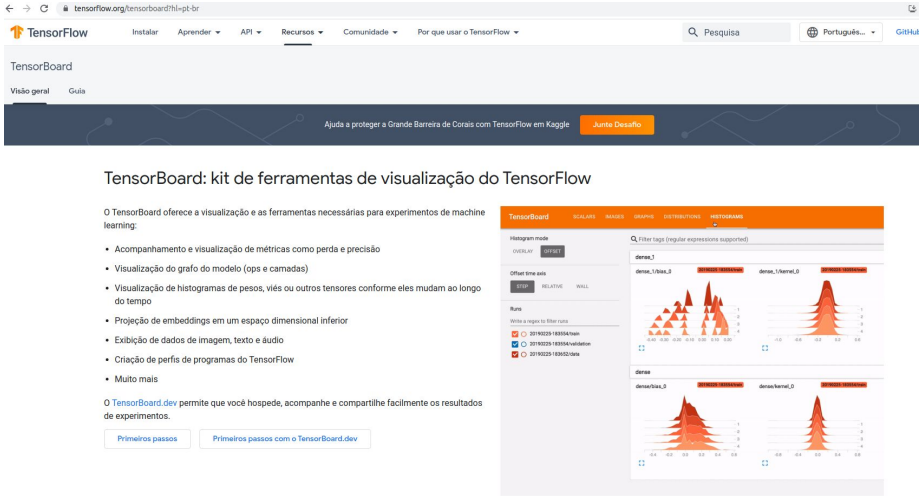
- Ciclo de ML





Atividade: (hiper)parametrização + Tensorboard

Depurar/experimentar o código (fashion_parametrizacao.ipynb) disponível no moodle com o tensorboard!
https://colab.research.google.com/drive/1XfnX4XhK5hnCdNmUgu6VV3MnSgWym1VQ?usp=sharing



Referências

[1] Aurélien Géron. Neural networks and deep learning https://www.oreilly.com/library/view/neural-networks-and/9781492037354/ch01.html

[2] François Chollet. Deep Learning with Python. Manning - Author do Keras!

Desbalanceamento de Classes

You could simply implement the class_weight from sklearn:

1. Let's import the module first
   from sklearn.utils import class_weight
2. In order to calculate the class weight do the following
   class_weights = class_weight.compute_class_weight(class_weight='balanced', classes=np.unique(y_train), y= y_train)
3. `class_weights=dict(enumerate(class_weights))`
4. Thirdly and lastly add it to the model fitting
   model.fit(X_train, y_train, class_weight=class_weights)

dicas

https://machinelearningmastery.com/improve-deep-learning-performance/